AI Predictive Maintenance & Root Cause Analysis Platform

Lean 10-Day Build Plan — 2 hours/day (~20 hours total)

Built for a resume/portfolio project: every technology in the original stack is touched (PyTorch, LSTM, XGBoost, FastAPI, PostgreSQL, Docker, MLflow), but scoped down so it's actually finishable in 20 hours instead of the 60-80 a full version would take.

What's simplified vs. the full version (and why it's still resume-solid)

Full-scope item	Lean version	Why this is fine
2 datasets (C-MAPSS + AI4I)	**C-MAPSS only**	It's a recognized benchmark; still supports RUL + failure prediction
Separate failure classifier	**RUL threshold → failure flag** (e.g. RUL < 20 = "failing soon")	One model, two outputs — still a legitimate design choice you can explain
Optuna hyperparameter sweep	**Sensible defaults + 1 manual tweak**	Tuning depth isn't what gets you the interview
Alembic migrations	`SQLAlchemy.create_all()`	Same DB result, way less setup
Full pytest suite	**2-3 smoke tests**	Enough to prove the endpoints work
Multi-page dashboard	**Single-page Streamlit view**	Still visually demoable

Keep this table in your back pocket — "I scoped this deliberately for a 20-hour build, here's what I'd add for production" is a strong interview answer.

Repo structure (set up Day 1)

```
predictive-maintenance/
├── data/{raw,processed}/
├── src/
│   ├── data/          # preprocessing.py
│   ├── models/        # xgboost_model.py, lstm.py
│   ├── rca/           # root_cause.py (SHAP + recommendation lookup)
│   ├── api/           # main.py, schemas.py
│   └── db/            # models.py (SQLAlchemy)
├── dashboard/         # streamlit_app.py
├── docker/            # Dockerfile.api, docker-compose.yml
├── requirements.txt
└── README.md
```

Day 1 (2h) — Setup & Data

- [ ] Repo + folder structure, venv, `requirements.txt`: `torch xgboost fastapi uvicorn sqlalchemy psycopg2-binary mlflow pandas numpy scikit-learn shap streamlit`
- [ ] Download NASA C-MAPSS **FD001** subset only (smallest, cleanest — don't grab all 4 sub-datasets)
- [ ] Load into pandas, understand columns (unit, cycle, 3 op settings, 21 sensors)
- [ ] Compute RUL label: `RUL = max_cycle_for_unit - current_cycle`, capped at 125

Deliverable: data loaded, RUL column computed, columns understood.

Day 2 (2h) — Feature Engineering

- [ ] Rolling mean + std per sensor (one window size, e.g. 10 cycles — don't sweep multiple)
- [ ] Train/val split **by unit ID** (critical — never split mid-sequence)
- [ ] Save processed train/val to `data/processed/` as CSV or parquet

Deliverable: `preprocessing.py`, processed features ready for both models.

Day 3 (2h) — XGBoost Model (your primary model)

- [ ] Train `XGBRegressor` on engineered features → predict RUL
- [ ] Evaluate: RMSE/MAE on validation set
- [ ] Derive failure flag: `predicted_RUL < 20` → `failing_soon = True`
- [ ] Save model with `joblib`

Deliverable: working XGBoost RUL model + threshold-derived failure prediction, one metric you can quote (e.g. "RMSE of X cycles").

Day 4 (2h) — LSTM Model (comparison model, kept lightweight)

- [ ] PyTorch `Dataset` for sliding windows (one sequence length, e.g. 30 cycles — no sweep)
- [ ] Small 1-2 layer LSTM → dense → RUL output
- [ ] Train fixed number of epochs (e.g. 30-50) with early stopping, save checkpoint
- [ ] Compare RMSE against XGBoost — write the number down, that's your "I compared two approaches" line

Deliverable: trained LSTM checkpoint + one-line comparison vs XGBoost.

Day 5 (2h) — MLflow + Root Cause Analysis

- [ ] Add `mlflow.log_params/log_metrics` to both training scripts, log both runs (retroactive re-run is fine)
- [ ] SHAP `TreeExplainer` on the XGBoost model → get top-3 contributing features per prediction
- [ ] `src/rca/root_cause.py`: function that takes a prediction + SHAP values → returns ranked top factors

Deliverable: two runs visible in the MLflow UI, working `get_root_cause()` function returning top-3 features.

Day 6 (2h) — Recommendations + FastAPI skeleton

- [ ] Small lookup dict/table: `{dominant sensor/feature → likely cause → recommended action}` — 8-10 entries is enough (e.g. "sensor 4 sustained rise → bearing wear → schedule inspection")
- [ ] Start FastAPI app: `POST /predict` — loads XGBoost model, returns `{RUL, failing_soon, top_factors, root_cause, recommendation}`
- [ ] Test via `/docs` (Swagger)

Deliverable: FastAPI running locally, `/predict` returns a full structured response in one call.

Day 7 (2h) — PostgreSQL + Alerts

- [ ] Minimal schema via SQLAlchemy: `machines`, `predictions`, `alerts` (3 tables, no more)
- [ ] `create_all()` to stand up tables — skip migrations tooling
- [ ] Wire `/predict` to write a `predictions` row, and auto-create an `alerts` row when `failing_soon = True`
- [ ] Add `GET /alerts` endpoint

Deliverable: Postgres persisting predictions + alerts, verified with a couple of test calls.

Day 8 (2h) — Dashboard

- [ ] Single-page Streamlit app: dropdown to pick a machine/unit → shows RUL, failure-risk indicator, top-3 root cause factors as a bar chart, and the current alert list (pulled from Postgres or via the API)
- [ ] Keep it to one page — don't build multi-tab navigation, it's not worth the time here

Deliverable: working dashboard, screenshot-able for your resume/portfolio.

Day 9 (2h) — Docker

- [ ] `Dockerfile.api` for the FastAPI service
- [ ] `docker-compose.yml`: Postgres + API (+ MLflow if time allows; run the dashboard locally with `streamlit run` if Docker runs long — it's not worth losing the whole day to)
- [ ] Get `docker-compose up` working end-to-end from a clean clone

Deliverable: containerized API + DB stack, one working `docker-compose up`.

> Docker is the most likely day to overrun — if you're short on time, containerizing just the API + Postgres and running the dashboard/MLflow locally is a completely reasonable trim, and easy to explain in an interview.

Day 10 (2h) — README, Polish, Demo Prep

- [ ] README: architecture diagram (text/ASCII is fine), setup steps, sample API call + response, screenshot of the dashboard
- [ ] Push to GitHub with clean commits
- [ ] Prepare a 2-minute live-demo script: ingest → predict → show root cause → show alert → show dashboard
- [ ] Write 2-3 resume bullet points (see below)

Deliverable: GitHub repo link ready to put on your resume, demo script rehearsed.

Resume bullet point starters (fill in your real numbers)

- *Built an end-to-end predictive maintenance platform combining LSTM and XGBoost models to forecast Remaining Useful Life on the NASA C-MAPSS turbofan dataset, achieving an RMSE of __ cycles.*
- *Implemented SHAP-based root cause analysis to explain individual failure predictions, mapping top contributing sensors to actionable maintenance recommendations.*
- *Designed and deployed a FastAPI + PostgreSQL backend with MLflow experiment tracking, containerized via Docker Compose, serving predictions through a Streamlit analytics dashboard.*

If you fall behind

Priority order to protect (don't cut these even under time pressure):

1. XGBoost model + RCA (this is your core differentiator and easiest to explain)
2. FastAPI `/predict` endpoint
3. Postgres persistence
4. Docker Compose (even a minimal 2-service version)

Cut in this order if a day runs long: LSTM depth (keep it, but don't tune it further) → dashboard polish → MLflow (log runs manually if the server setup eats time).